

Fundamentals of Data Structures

Fundamentals of Data Structures

College of science – Department of computer science

Author - Danish Manzoor Ahmad

Date – 15 November 2025

Data structures

Introduction - Data structures are fundamental to computer science, defining how data is organized and stored for efficient access and modification. In C++, we have both fundamental data structures and Standard Template Library (STL) containers that implement these concepts. This document covers four core linear data structures: Arrays, Lists, Stacks, and Queues, detailing their C++ implementation and common algorithmic applications.

Data structures are introduced in order to store, organize and manipulate data in programming languages. They are designed in a way that makes accessing and processing of the data a little easier and simpler. These data structures are not confined to one particular programming language; they are just pieces of code that structure data in the memory.

Data types are often confused as a type of data structures, but it is not precisely correct even though they are referred to as Abstract Data Types. Data types represent the nature of the data while data structures are just a collection of similar or different data types in one.

There are usually just two types of data structures –

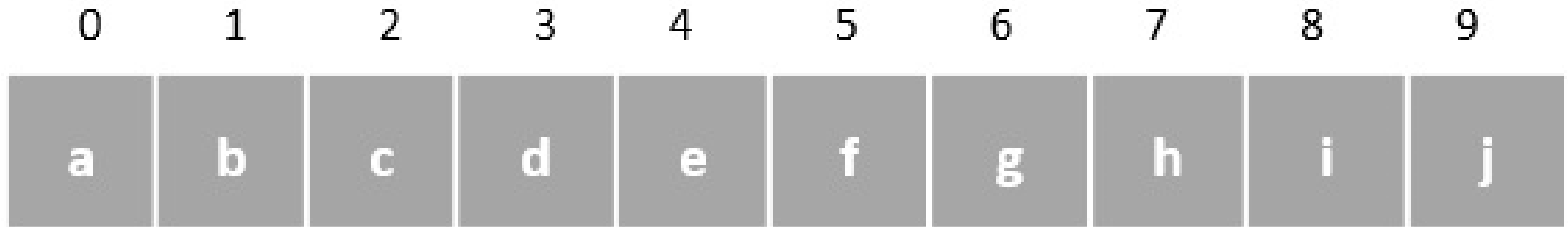
There are usually just two types of data structures –

Linear

Non-Linear

Linear Data Structures

The data is stored in linear data structures sequentially. These are rudimentary structures since the elements are stored one after the other without applying any mathematical operations.



Static Linear Data Structures And Dynamic Linear Data Structures

Static Linear Data Structures - In Static Linear Data Structures, the memory allocation is not scalable. Once the entire memory is used, no more space can be retrieved to store more data. Hence, the memory is required to be reserved based on the size of the program. This will also act as a drawback since reserving more memory than required can cause a wastage of memory blocks.

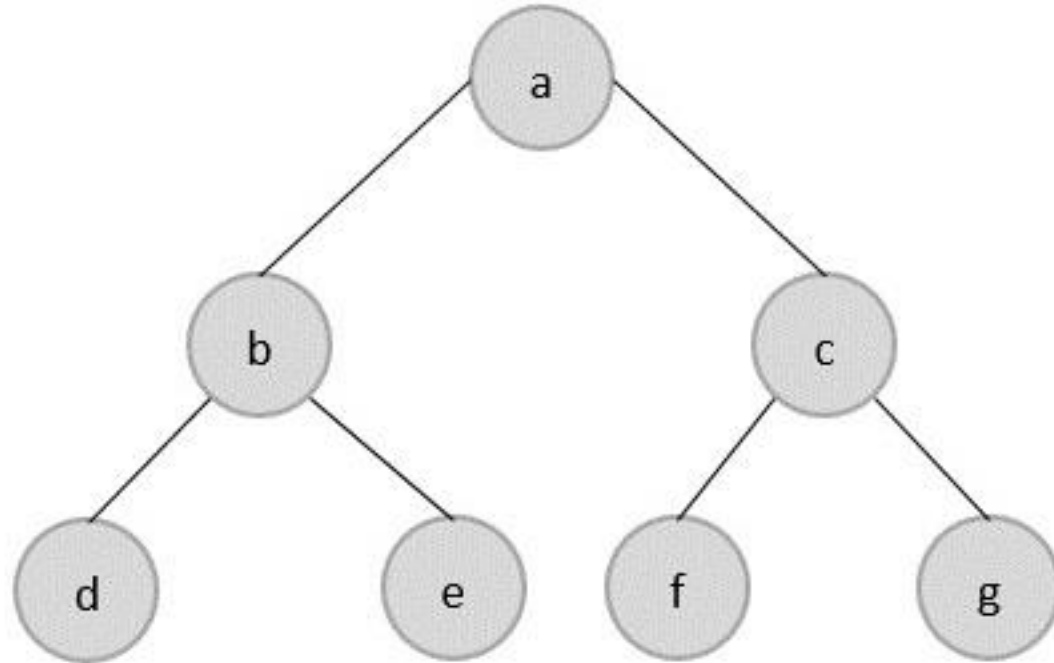
The best example for static linear data structures is an array.

Dynamic Linear Data Structures - In Dynamic linear data structures, the memory allocation can be done dynamically when required. These data structures are efficient considering the space complexity of the program.

Few examples of dynamic linear data structures include: linked lists, stacks and queues.

Non-Linear Data Structures

Non-Linear Data Structures - Non-Linear data structures store the data in the form of a hierarchy. Therefore, in contrast to the linear data structures, the data can be found in multiple levels and are difficult to traverse through.



Non-Linear Data Structures

However, they are designed to overcome the issues and limitations of linear data structures. For instance, the main disadvantage of linear data structures is the memory allocation. Since the data is allocated sequentially in linear data structures, each element in these data structures uses one whole memory block. However, if the data uses less memory than the assigned block can hold, the extra memory space in the block is wasted. Therefore, non-linear data structures are introduced. They decrease the space complexity and use the memory optimally.

Few types of non-linear data structures are –

Graphs

Trees

Tries

Maps

1. Arrays

Definition

An array is a contiguous collection of elements of the same type, accessible via indices. In C++, arrays can be static (fixed-size) or dynamic.

C++ Implementation

Static Arrays: Fixed size, stack-allocated

Dynamic Arrays: Variable size, heap-allocated using `new`

Overview of Arrays

What is an Array?

Array Notation in C++

Array Rank

Accessing Array Elements

Checking Array Bounds

Comparing Arrays to Collections

1. Arrays

Creating Arrays

Creating Array Instances

Initializing Array Elements

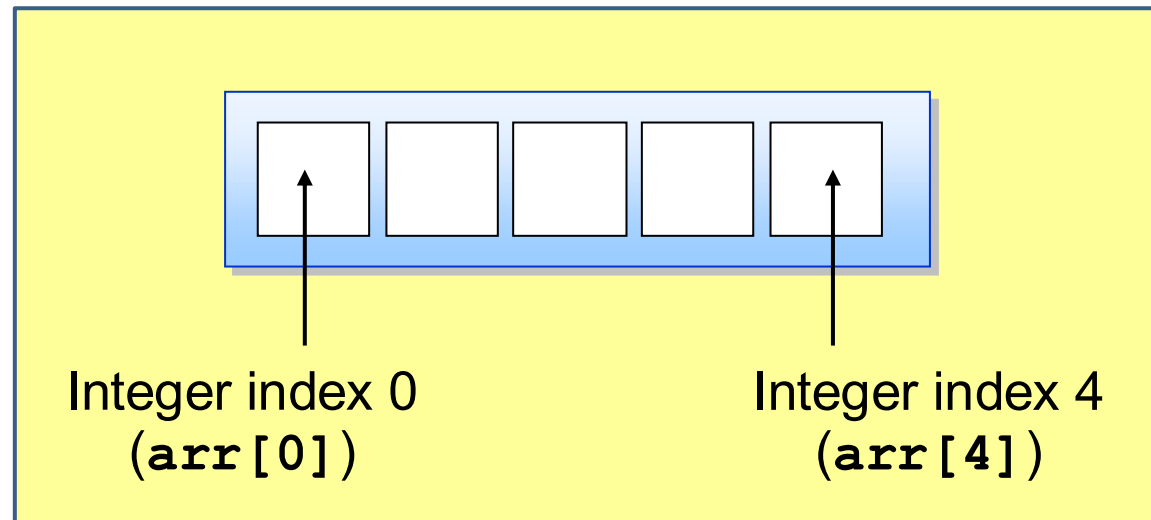
Initializing Multidimensional Array Elements

Creating a Computed Size Array

Copying Array Variables

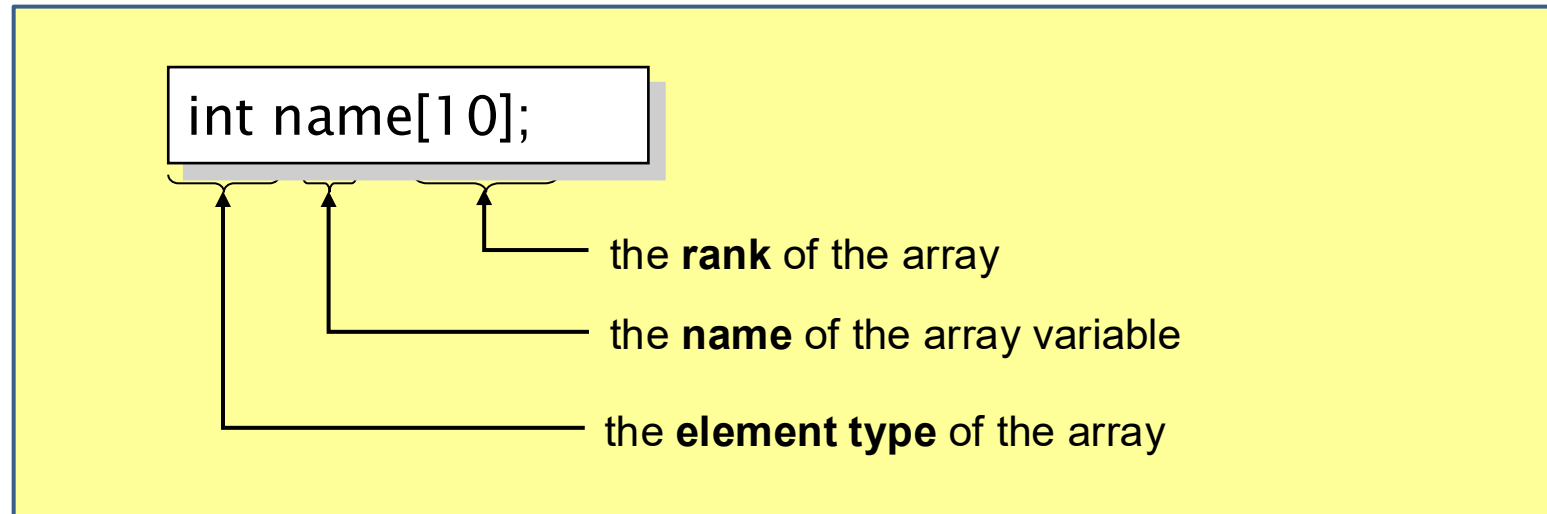
What is an Array?

- An array is a sequence of homogeneous elements.
 - All elements in an array have the same type.
 - Individual elements are accessed using integer indexes.
- Structs can have elements of different types.



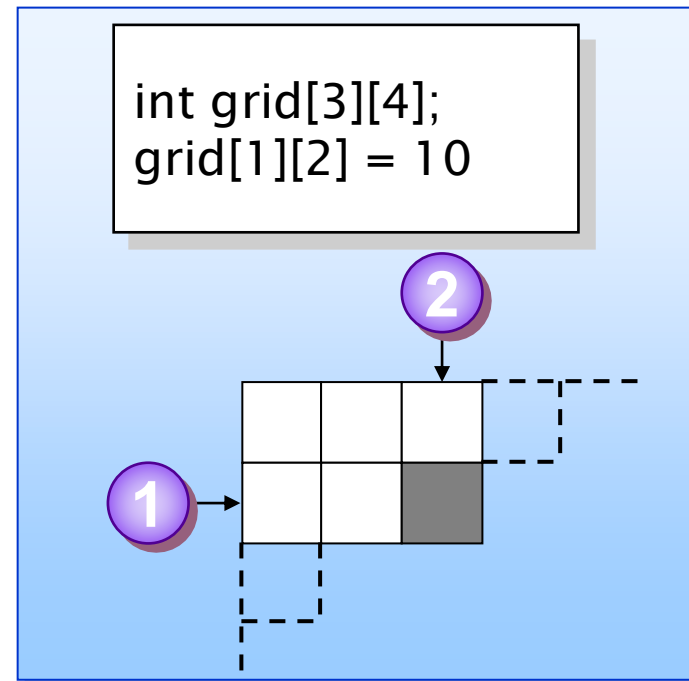
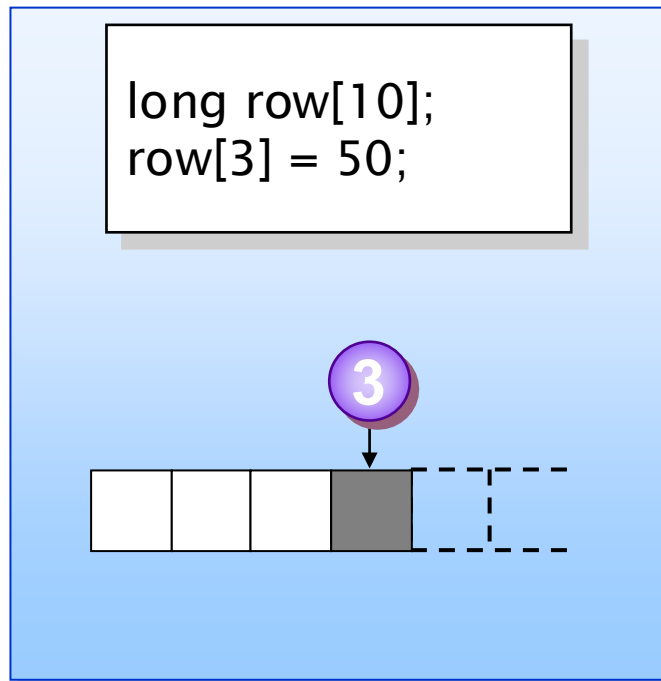
Array Notation in C++

- You declare an array variable by specifying:
 - The **element type** of the array.
 - The **rank** of the array.
 - The **name** of the array variable.



Accessing Array Elements

- Supply an integer index for each rank.
- Indexes are zero-based.



Code Examples:

Code Examples:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
// Static array example
```

```
void staticArrayExample() {
```

```
    int staticArr[5] = {1, 2, 3, 4, 5};
```

```
    cout << "Static array: ";
```

```
    for(int i = 0; i < 5; i++) {
```

```
        cout << staticArr[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
}
```

// Dynamic array example

```
void dynamicArrayExample() {  
    int size = 5;  
    int* dynamicArr = new int[size];  
    for(int i = 0; i < size; i++) {  
        dynamicArr[i] = (i + 1) * 10;  
    }  
    cout << "Dynamic array: ";  
    for(int i = 0; i < size; i++) {  
        cout << dynamicArr[i] << " ";  
    }  
    cout << endl;  
    delete[] dynamicArr; // Don't forget to free memory!  
}
```

Application in Algorithms // Binary Search using arrays

```
int binarySearch(const vector<int>& arr, int target) {  
    int left = 0;  
    int right = arr.size() - 1;  
  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
        if (arr[mid] == target) return mid;  
        else if (arr[mid] < target) left = mid + 1;  
        else right = mid - 1;  
    }  
    return -1; // Not found  
}
```

2. Lists (Linked Lists)

Definition

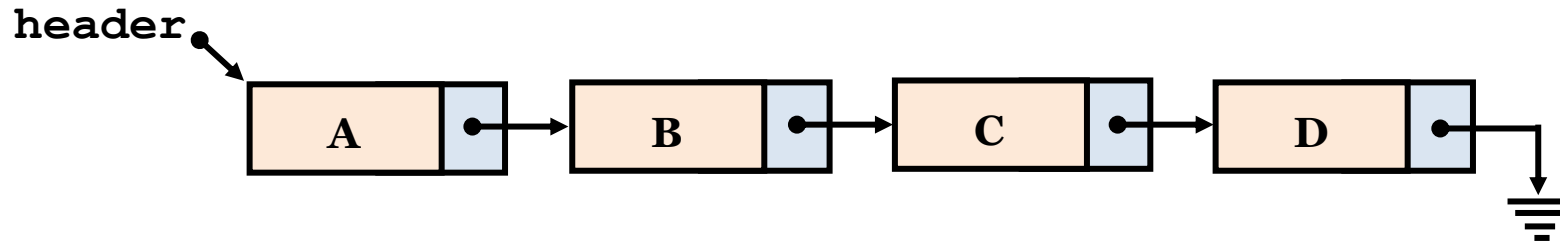
A linked list is a linear collection of nodes where each node contains data and a pointer to the next node. C++ provides `std::list` as a doubly-linked list implementation.

C++ Implementation

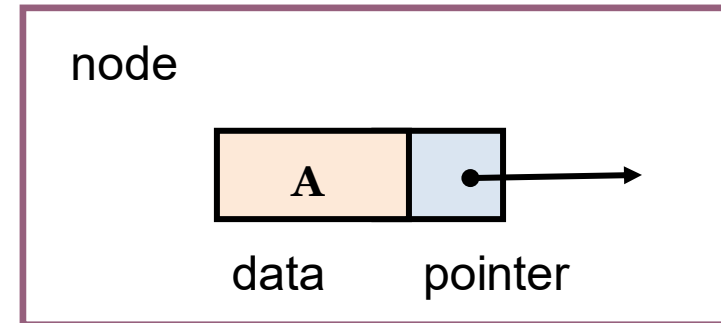
Manual Implementation: Create Node class and manage pointers

STL List: Doubly-linked list with efficient insertion/deletion

Linked List (Singly Linked List)

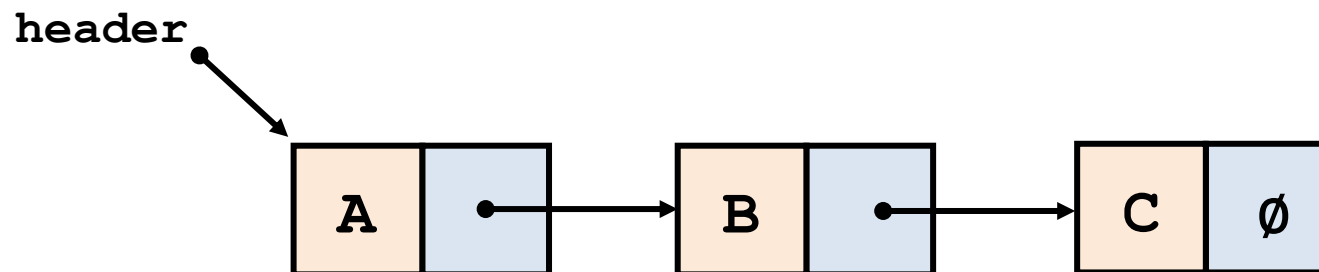


- **Linked List:** is a series of connected *nodes*.
- Each *node* contains:
 - At least a piece of **data** (any type).
 - **Pointer** to the next node in the list.
- **Header:** pointer to the first node.
- The last node points to **NULL**.



Linked List (Singly Linked List)

- Linked List is a data structure which can change its size during execution (dynamic allocation).
 - Successive elements are connected by pointers (references).
 - It can grow or shrink in size during execution of a program.
 - It can be made just as long as required.
 - It does not waste memory space.



Code Examples:

Code Examples:

```
#include <iostream>
```

```
#include <list>
```

```
using namespace std;
```

```
// Manual singly linked list implementation
```

```
class ListNode {
```

```
public:
```

```
    int data;
```

```
    ListNode* next;
```

```
    ListNode(int val) : data(val), next(nullptr) {}
```

```
};
```

```
class LinkedList {
```

```
private:
```

```
    ListNode* head;
```

```
public:
```

```
    LinkedList() : head(nullptr) {}
```

```
    void insertAtEnd(int val) {
```

```
        ListNode* newNode = new ListNode(val);
```

// STL list example

```
void stlListExample() {  
    list<int> myList = {1, 2, 3, 4, 5};  
    // Add elements  
    myList.push_back(6);  
    myList.push_front(0);  
    // Insert in middle  
    auto it = myList.begin();  
    advance(it, 3); // Move iterator to position 3  
    myList.insert(it, 99);  
    // Remove elements  
    myList.pop_back();  
    myList.pop_front();  
    cout << "STL List: ";  
    for(int num : myList) {  
        cout << num << " ";  
    }  
    cout << endl;  
    cout << "Size: " << myList.size() << endl;  
}
```

Application in Algorithms // Detecting cycle in linked list (Floyd's Cycle Detection)

```
bool hasCycle(ListNode* head) {  
    if (!head || !head->next) return false;  
  
    ListNode* slow = head;  
    ListNode* fast = head;  
  
    while (fast && fast->next) {  
        slow = slow->next;  
        fast = fast->next->next;  
        if (slow == fast) return true;  
    }  
    return false;  
}
```

// Reversing a linked list

```
ListNode* reverseList(ListNode* head) {  
    ListNode* prev = nullptr;  
    ListNode* current = head;  
    while (current) {  
        ListNode* nextTemp = current->next;  
        current->next = prev;  
        prev = current;  
        current = nextTemp;  
    }  
    return prev;  
}
```

3. Stacks

Definition

A stack follows Last-In-First-Out (LIFO) principle. Elements are added and removed from the top only.

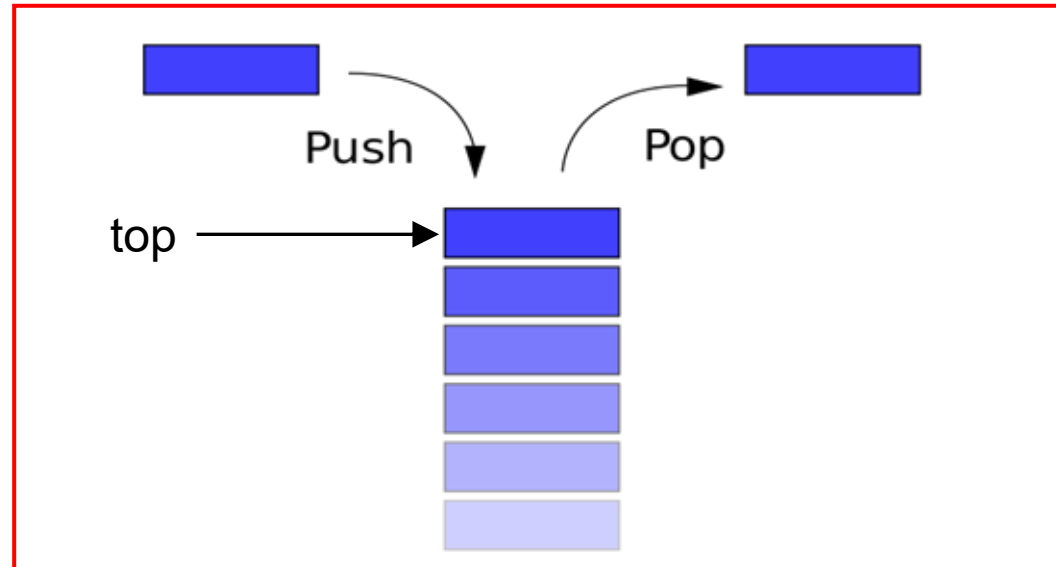
C++ Implementation

Using Arrays/Vectors: Manual implementation

STL Stack: Container adapter that can use various underlying containers

Stack

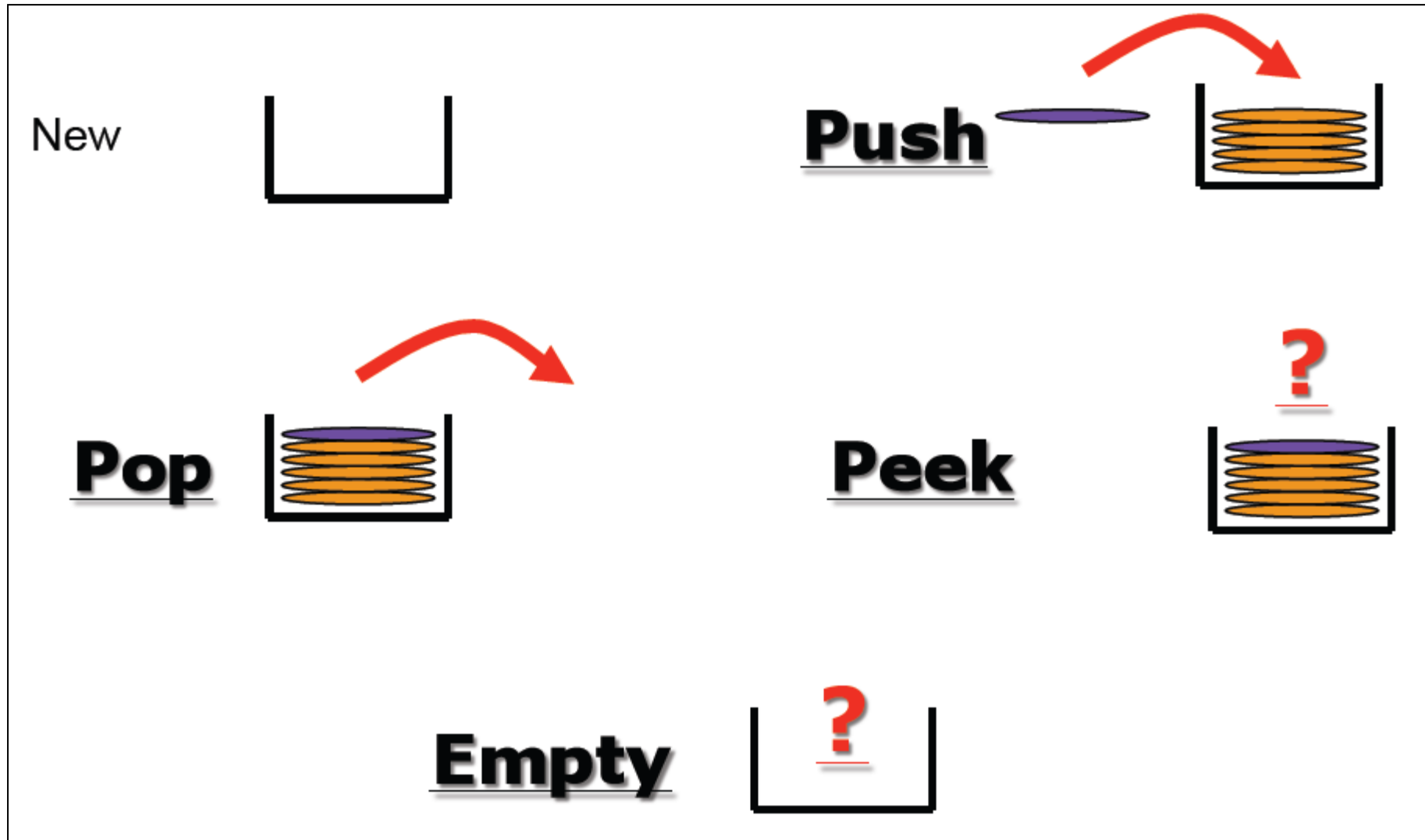
- **Stack:** is an abstract data type (ADT) in which elements are added and removed from only one end (the **top**).
- Stack is a **LIFO** “Last In, First Out” structure.
 - The last element inserted will be the first to be retrieved.



Stack Operations

Push	Adds new item to the top of the stack. (throws exception if stack is full)
Pop	Removes the item at the top of the stack. (throws exception if stack is empty)
MakeEmpty	Sets stack to an empty state.
IsEmpty	Determines whether the stack is currently empty.
IsFull	Determines whether the stack is currently full. (only for stacks that implemented using Arrays)
Top	Returns a copy of the top item. (throws exception if stack is empty)

Basic Operations of a Stack



Code Examples:

```
#include <iostream>
#include <stack>
#include <vector>
using namespace std;
// Manual stack using vector
class ManualStack {
private:
    vector<int> data;
public:
    void push(int val) {
        data.push_back(val);
    }
    void pop() {
        if (!isEmpty()) {
            data.pop_back();
        }
    }
    int top() {
```

// STL stack example

```
void stlStackExample() {  
    stack<int> st;  
    // Push elements  
    st.push(10);  
    st.push(20);  
    st.push(30);  
    cout << "Top element: " << st.top() << endl;  
    cout << "Stack size: " << st.size() << endl;  
  
    // Pop elements  
    st.pop();  
    cout << "After pop, top element: " << st.top() << endl;  
    // Check if empty  
    cout << "Is stack empty? " << (st.empty() ? "Yes" : "No") << endl;  
}
```

Application in Algorithms // Balanced parentheses checker

```
bool isBalancedParentheses(const string& expression) {  
    stack<char> st;  
    for (char ch : expression) {  
        if (ch == '(' || ch == '[' || ch == '{') {  
            st.push(ch);  
        } else if (ch == ')' || ch == ']' || ch == '}') {  
            if (st.empty()) return false;  
            char top = st.top();  
            st.pop();  
            if ((ch == ')' && top != '(') ||  
                (ch == ']' && top != '[') ||  
                (ch == '}' && top != '{')) {  
                return false;  
            }  
        }  
    }  
    return st.empty();  
}
```

4. Queues

Definition

A queue follows First-In-First-Out (FIFO) principle. Elements are added at the rear and removed from the front.

C++ Implementation

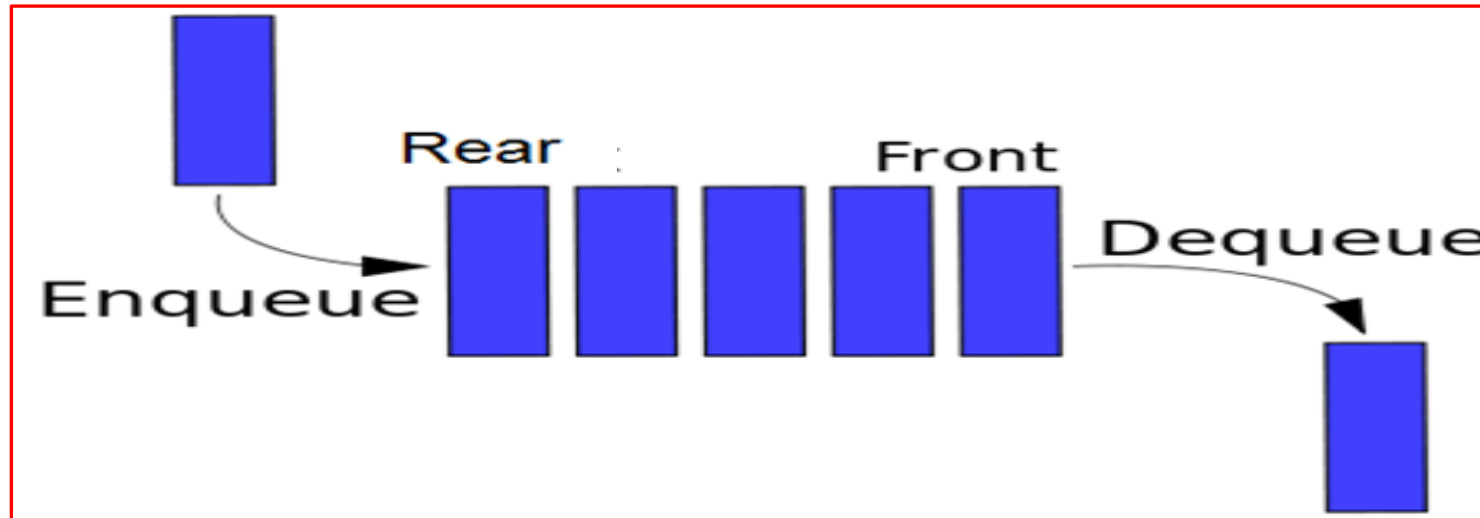
Using Arrays/Lists: Manual implementation

STL Queue: Container adapter

STL Deque: Double-ended queue

Queue

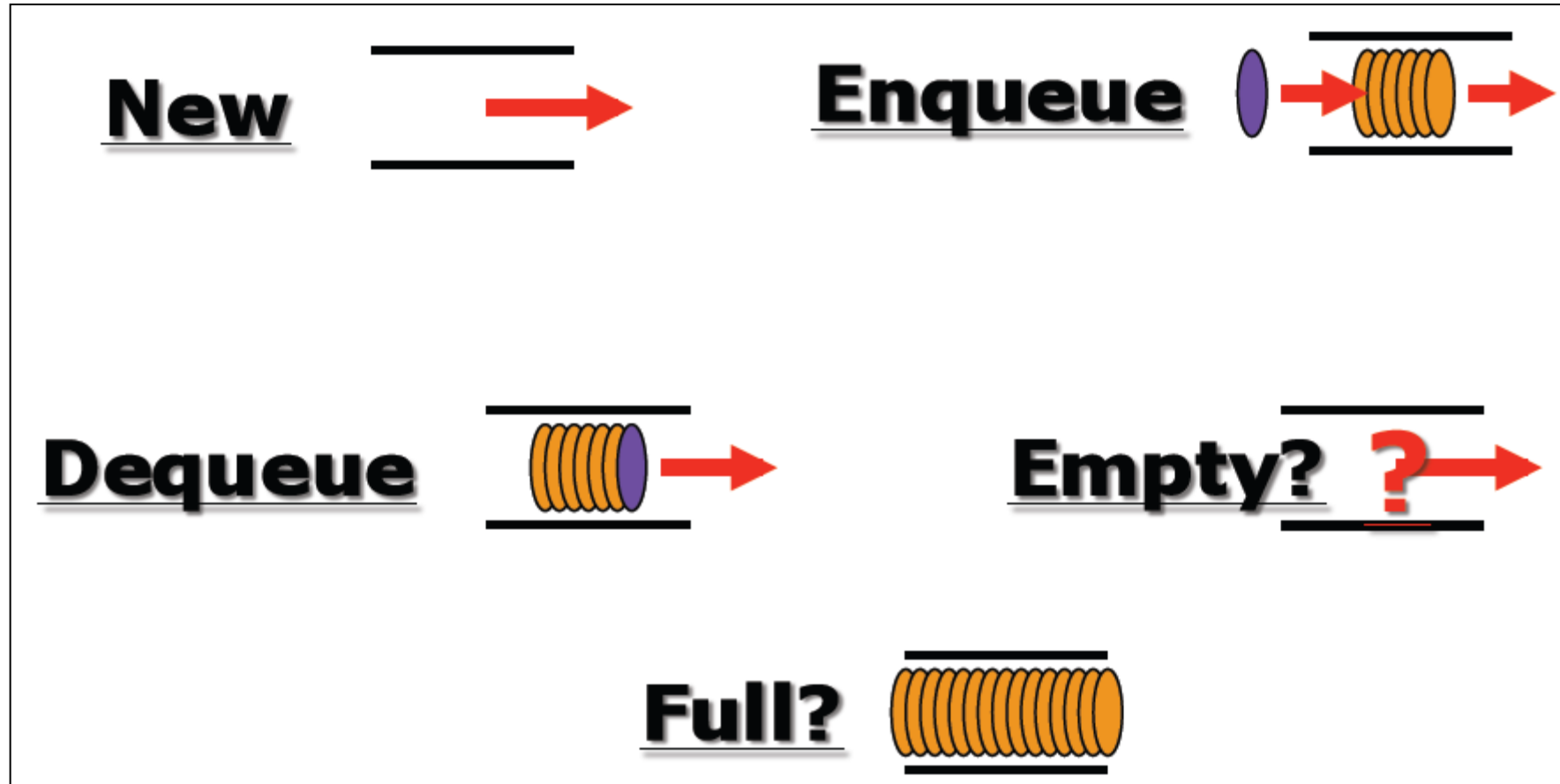
- **Queue**: is an abstract data type (ADT) in which items are added at one end (the **rear**) and removed from the other end (the **front**).
- Queue is a **FIFO** “First In, First Out” structure.
 - The first item in is the first item served (out).



Queue Operations

Enqueue	Adds new item to the rear of the queue.
Dequeue	Removes the item at the front of the queue.
MakeEmpty	Sets queue to an empty state.
IsEmpty	Determines whether the queue is currently empty.
IsFull	Determines whether the queue is currently full.

Basic Operations of a Queue



Code Examples:

```
#include <iostream>
#include <queue>
#include <deque>
using namespace std;
// Manual queue using linked list
class QueueNode {
public:
    int data;
    QueueNode* next;
    QueueNode(int val) : data(val), next(nullptr) {}
};
class ManualQueue {
private:
    QueueNode* front;
    QueueNode* rear;
public:
    ManualQueue() : front(nullptr), rear(nullptr) {}
```

// STL queue example

```
void stlQueueExample() {  
    queue<int> q;  
    // Enqueue elements  
    q.push(10);  
    q.push(20);  
    q.push(30);  
    cout << "Front element: " << q.front() << endl;  
    cout << "Back element: " << q.back() << endl;  
    cout << "Queue size: " << q.size() << endl;  
  
    // Dequeue elements  
    q.pop();  
    cout << "After dequeue, front element: " << q.front() << endl;  
  
    // Check if empty  
    cout << "Is queue empty? " << (q.empty() ? "Yes" : "No") << endl;  
}
```

// STL deque example

```
void dequeExample() {  
    deque<int> dq;  
    // Add elements to both ends  
    dq.push_back(10);  
    dq.push_front(5);  
    dq.push_back(15);  
    dq.push_front(1);  
    cout << "Deque: ";  
    for(int num : dq) {  
        cout << num << " ";  
    }  
    cout << endl;  
    // Remove from both ends  
    dq.pop_front();  
    dq.pop_back();  
    cout << "After operations: ";  
    for(int num : dq) {  
        cout << num << " ";  
    }
```

Application in Algorithms // Breadth-First Search (BFS) using queue

```
#include <vector>
#include <queue>
using namespace std;
void BFS(vector<vector<int>>& graph, int startNode) {
    int n = graph.size();
    vector<bool> visited(n, false);
    queue<int> q;
    visited[startNode] = true;
    q.push(startNode);
    cout << "BFS traversal: ";
    while (!q.empty()) {
        int current = q.front();
        q.pop();
        cout << current << " ";
        for (int neighbor : graph[current]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
```

Summary Table

Data Structure	C++ Implementation	Time Complexity	Key Applications
Array	int arr[N], std::vector	Access: O(1), Insert/Delete: O(n)	Binary Search, Dynamic Programming
List	std::list, Manual Node class	Insert/Delete: O(1), Access: O(n)	Cycle Detection, Memory Management
Stack	std::stack, Manual with vector	Push/Pop: O(1)	Parentheses Matching, DFS, Undo Operations
Queue	std::queue, std::dequeue	Enqueue/Dequeue: O(1)	BFS, Task Scheduling, Buffers

Summary : Key Concepts Summary

Question/Answer

Thank You